

j

#

Date: 26-03-2026

OOPS

class

class is a blueprint of object and

logical entity

class can create multiple objects

Objects

object is an instance of class

physical entity

#

self refer to the current object

"""

class Aizen:

def detail(self):

print("Yokoso")

print(id(self))

x=Aizen()

here x is an object and it has same id as self

x.detail()

print(id(x))

y=Aizen()

y.detail()

print(id(y))

"""

not working

"""

class A:

def func(self,name):

return name

x=A()

print(x.func())

print(A().__dict__)

"""

#

"""

class A:

def func(self,name):

return name

x=A()

print(x.func("Aizen"))

"""

#

"""

class A:

def func(self,name):

self.name=name

```

def display(self):
    print(self.name)
    x=A()
    x.func("Gin")
    x.display()      # it is the one call function to print Gin
    print(x.__dict__)
"""

```

constructor

```

# __init__ -- to create a constructor.
## init method is a special method in python which is used to
## --initialize the object of the class.
"""

```

```

class A:
    def __init__(self,name):
        print(name)
        A("Gin")
        #x=A("Ichimaru")
"""

```

```

# if __init__ there we don't need to call the function to the read def
"""

```

```

class A:
    def __init__(self,name):
        self.name=name
    def display(self):
        print(self.name)
        x=A("Ruken")
        x.display()      # it is the one call function to print Ruken
        print(x.__dict__)
"""

```

#

```

#                                     Date: 27-03-2026

```

```

# eg for next one
"""

```

```

class A:
    def func(self,a,b):
        print(a+b)
        x=A()
        x.func(10,20)
"""

```

```

# calling outside def variable -- (self.z)
"""

```

```

class A:
    z=30
    def func(self,a,b):
        print(a+b+self.z)

```

```

x=A()
x.func(10,20)
'''
# calling func arguments in another display function
'''

```

```

class A:
def func(self,a,b):
self.a=a
self.b=b
def display(self):
print(self.a+self.b)
x=A()
x.func(10,20)
x.display()
'''

```

```

# same but using init method
'''

```

```

class A:
def __init__(self,a,b):
self.a=a
self.b=b
def display(self):
print(self.a*self.b)
x=A(10,20)
x.display()
'''

```

```

# without argument in init but also same
'''

```

```

class A:
def __init__(self):
self.a=10
self.b=20
def display(self):
print(self.a*self.b)
x=A()
x.display()
'''

```

```

# destructor
# init is create the object and del is used to delete the object
'''

```

```

class A:
def __init__(self):
print("created")
def __del__(self):
print("deleted")
x=A()
del x
'''

```

```

# same but inverse the place
'''

```

```
class A:
def __del__(self):
print("deleted")
def __init__(self):
print("created")
x=A()
del x
'''
```

```
#
```

```
# Inheritance
```

```
# Single inheritance:
# class is derived from single parent class
```

```
'''
```

```
class dad:
def cash(self):
print("dad's cash")
class son(dad):
def bike(self):
print("son's bike")
d=dad()
d.cash()
```

```
s=son()
s.bike()
s.cash()
'''
```

```
# Multiple inheritance:
```

```
# class is derived from two or more parent class
'''
```

```
class dad:
def cash(self):
print("dad's cash")
class mom:
def phone(self):
print("mom's phone")
class son(dad,mom):
def bike(self):
print("son's bike")
```

```
d=dad()
d.cash()
```

```
m=mom()
m.phone()
```

```
s=son()
s.bike()
s.cash()
```

```
s.phone()
```

```
'''
```

```
# Multilevel inheritance:
```

```
# a class is derived from a derived class which is also derived from a parent class
```

```
'''
```

```
class grand:
def land(self):
print("grandpa's land")
class dad(grand):
def cash(self):
print("dad's cash")
```

```
class son(dad):
def bike(self):
print("son's bike")
```

```
g=grand()
g.land()
```

```
d=dad()
d.cash()
d.land()
```

```
s=son()
s.bike()
s.cash()
s.land()
```

```
'''
```

```
# hierarchical inheritance:
```

```
# two or more class is derived from a single parent class
```

```
'''
```

```
class dad:
def cash(self):
print("dad's cash")
class son(dad):
def bike(self):
print("son's bike")
class daughter(dad):
def gold(self):
print("daughter's ")
```

```
d=dad()
d.cash()
```

```
s=son()
s.bike()
s.cash()
```

```
d=daughter()
d.gold()
d.cash()
```

'''

hybrid inheritance

using two or more inheritance

'''

```
class dad:
    def cash(self):
        print("dad's cash")
class mom:
    def phone(self):
        print("mom's phone")
class daughter(dad,mom):
    def gold(self):
        print("daughter's gold")
class sonInLaw(daughter):
    def house(self):
        print("sonInlaw's house")
```

```
d=dad()
d.cash()
```

```
m=mom()
m.phone()
```

```
d=daughter()
d.gold()
d.cash()
d.phone()
```

```
s=sonInLaw()
s.house()
s.gold()
s.cash()
s.phone()
'''
```

#

Date: 28-03-2026

saturday -- assessment and mock

#

Date: 29-03-2026

sunday -- Holiday

#

#

Date: 30-03-2026

mro -- Method Resolution order

c3 linearization

mro of F = FDBECAO

'''

```
class A:
def __init__(self):
print("A")
class B(A):
def __init__(self):
print("B")
class C(A):
def __init__(self):
print("C")
class D(B):
def __init__(self):
print("D")
class E(C):
def __init__(self):
print("E")
class F(D,E):
def __init__(self):
print("E")
```

```
f=F()
print(F.__mro__)
'''
```

mro of F = FDEBCAO

'''

```
class A:
def __init__(self):
print("A")
class B(A):
def __init__(self):
print("B")
class C(A):
def __init__(self):
print("C")
class D(B,c):
def __init__(self):
print("D")
class E(B,C):
def __init__(self):
print("E")
class F(D,E):
```

```
def __init__(self):
    print("E")
```

```
f=F()
print(F.__mro__)
'''
```

```
# mro of F == FDBAECO
'''
```

```
class A:
    def __init__(self):
        print("A")
class B:
    def __init__(self):
        print("B")
class C:
    def __init__(self):
        print("C")
class D(B):
    def __init__(self):
        print("D")
class E(C):
    def __init__(self):
        print("E")
class F(D,A,E):
    def __init__(self):
        print("E")
```

```
f=F()
print(F.__mro__)
'''
```

```
# super(). method in single
```

```
# used for calling parent class method with same method name inside of a child class
'''
```

```
class Dad():
    def cash(self):
        print("Dad's cash")
class Son(Dad):
    def cash(self):
        super().cash()
        print("Son's cash")
d=Dad()
d.cash()
s=Son()
s.cash()
'''
```

```
# same but reverse order for calling
```

```
'''
class Dad():
    def cash(self):
        print("Dad's cash")
```



```

class Son(Dad):
def cash(self):
print("Son's cash")
super().cash()
d=Dad()
d.cash()
s=Son()
s.cash()
'''

```

```

# multiple inheritance of super()
'''

```

```

class Dad():
def cash(self):
print("Dad's cash")

```

```

class Mom():
def cash(self):
print("Mom's cash")
super().cash()
class Son(Mom,Dad):
def cash(self):
print("Son's cash")
super().cash()

```

```

s=Son()
s.cash()
'''

```

```

# multiple
'''

```

```

class Dad():
def cash(self):
print("Dad's cash")

```

```

class Mom():
def cash(self):
print("Mom's cash")

```

```

class Son(Mom,Dad):
def cash(self):
print("Son's cash")
super().cash()
Dad().cash()
s=Son()
s.cash()
'''

```

```

# multilevel
'''

```

```

class Grand():
def cash(self):
print("Grandpa's cash")

```

```

class Dad(Grand):

```

```
def cash(self):
    print("Dad's cash")
    super().cash()
```

```
class Son(Dad):
    def cash(self):
        print("Sons's cash")
        super().cash()
```

```
s=Son()
s.cash()
'''
```

```
# hierrchical
'''
```

```
class Dad():
    def cash(self):
        print("Dad's cash")
```

```
class Son(Dad):
    def cash(self):
        print("Sons's cash")
        super().cash()
```

```
class Daughter(Dad):
    def cash(self):
        print("daughter's cash")
        super().cash()
s=Son()
s.cash()
```

```
d=Daughter()
d.cash()
'''
```

```
# Hybrid
'''
```

```
class A:
    def __init__(self):
        print("A")
    class B(A):
        def __init__(self):
            super().__init__()
            print("B")
    class C(A):
        def __init__(self):
            print("C")
    class D(B):
        def __init__(self):
            super().__init__()
            print("D")
    class E(C):
        def __init__(self):
            print("E")
```

```
class F(D,E):
def __init__(self):
super().__init__()
print("F")
f=F()
'''
```

```
# op -- ABCDEF
'''
```

```
class A:
def __init__(self):
print("A")
class B(A):
def __init__(self):
super().__init__()
print("B")
class C(B):
def __init__(self):
super().__init__()
print("C")
class D(B):
def __init__(self):
super().__init__()
print("D")
class E(D):
def __init__(self):
super().__init__()
print("E")
class F(E,C):
def __init__(self):
super().__init__()
print("F")
f=F()
'''
```

```
#
```

```
#
```

Date: 31-03-2026

```
#
'''
```

```
class A:
def __init__(self,name):
print("Yo ",name)
class B:
def __init__(self):
print("B")
b=B()
'''
```

```
#
```

```
"""
class A:
def __init__(self,name):
print("yo ",name)
class B(A):
pass
b=B("Zoro")
"""
```

```
#
```

```
"""
class A:
def __init__(self,name):
print("yo ",name)
class B(A):
def __init__(self):
super().__init__()
class C(B):
def __init__(self):
super().__init__("Zoro")
c=C()
"""
```

```
#
```

```
"""
class A:
def __init__(self,name):
print("yo ",name)
class B(A):
def __init__(self):
print("Yokoso")
class C(B):
def __init__(self):
A.__init__(self,"Zoro")
c=C()
"""
```

```
# polymorphism:
```

```
# poly -- many
## morphism -- forms
```

```
# where one entity(method, operator, object) can take many forms,
#-allowing different types to be treated through a common superclass.
# +
```

```
# eg + can add and as well as concat
```

```
"""
a=20
b=10
```

```
print(a+b)
```

```
a="sakasama"  
b=" no sekai"
```

```
print(a+b)  
"""
```

```
#Method OverLoading -- compile time polymorphism
```

```
# same class and same method name but different parameters.
```

```
## python doesn't support this so we can achieve through importing multiple dispatch --  
@dispatch
```

```
# _____
```

```
# Method Overriding -- runtime polymorphism
```

```
#different class same method name and same parameters.  
## python supports this
```

```
# reference for method overloading  
"""
```

```
class A:  
def func(self,a,b):  
print(a+b)  
def func(self,a,b,c=0): # here it is the one running  
print(a+b+c)  
a=A()  
a.func(10,20)  
"""
```

```
#  
"""
```

```
class A:  
def func(self,*args):  
count=0  
for i in args:  
count=count+i  
print(count)  
a=A()  
a.func(10,20,30)  
"""
```

```
#
```

```
#      tuesday - leave
```

```
Date: 01-04-2026
```

```
#
```

#

Date: 02-04-2026

method overloading and overriding also in another file in vs code

Encapsulation:

Access Modifiers : public -- private __ -- protect _

public

"""

class A:

def func(self,x):

self.x=x

print(self.x)

a=A()

a.func(10)

"""

we can access entity in another class

"""

class A:

def __init__(self,x):

self.x=x

class B(A):

def display(self):

print(self.x)

b=B(7)

-- here only have to give argument

b.display()

"""

#

"""

class A:

def func(self,x):

self.x=x

a=A()

a.func(96)

print(a.x) # we can access in outside also public

"""

here we access inherits in another class and print is also public

"""

class A:

def func(self):

print("Yo")

class B(A):

def display(self):

self.func()

b=B()

b.display()

"""

private can access only its own class not other class

#__ double underscore is the private declaration

##but can access in other class calling the class name -- name manging method

```
#  
"""  
class A:  
def func(self,x):  
self.__x=x  
print(self.__x)  
a=A()  
a.func(18)  
"""
```

```
#  
"""  
class A:  
def __init__(self,x):  
self.__x=x  
def display(self):  
print(self.__x)  
a=A(10)  
a.display()  
"""
```

this is name manging

```
"""  
class A:  
def __init__(self,x):  
self.__x=x  
class B(A):  
def display(self):  
print(self._A__x)    # here if (self.__x) is error cuz its private  
b=B(10)  
b.display()  
"""
```

same name manging outside calling

```
"""  
class A:  
def __init__(self,x):  
self.__x=x  
class B(A):  
def display(self):  
print(self._A__x) # here self  
b=B(10)  
b.display()  
print(b._A__x) # here b  
"""
```

private method

```
"""  
class A:  
def __func(self):  
print("yokoso")  
class B(A):
```

```

def display(self):
self._A__func() # name mangling private method

b=B()
b.display()
b._B__func()
"""

# summa
"""

class __A:
def func(self):
print("sakasama no sekai")
class B(__A):
def display(self):
self.func()
b=B()
b.func()
"""

# protected: using -- _ single underscore -- naming conversion

#

# argument
"""

class A:
def func(self,x):
self._x=x
print(self._x)
a=A()
a.func(10)
print(a._x) # outside of the class argument
"""

# another method argument
"""

class A:
def func(self,x):
self._x=x
def display(self):
print(self._x)
a=A()
a.func(10)
a.display()
"""

# another class argument
"""

class A:
def __init__(self,x):
self._x=x
class B(A):
def display(self):
print(self._x)
b=B(10)
b.display()

```



```
"""
# method protected inside classss
"""
```

```
class A():
def _func(self):
print("Bankai")
def display(self):
self._func()
a=A()
a.display()
"""
```

```
# outside another class
"""
```

```
class A():
def _func(self):
print("Genjutsu")
class B(A):
def display(self):
self._func()
b=B()
b.display()
"""
```

```
#
```

```
#
```

Date: 03-04-2026

```
# eg (summa)
"""
```

```
class A:
def setname(self,a):
self.a=a
def getname(self):
print(self.a)
a=A()
a.setname("Zaraki")
a.getname()
"""
```

```
# Encapsulation
```

```
# which is used for protect the objets from outside changes
```

```
# @property - but using this we can change
```

```
"""
```

```
class A:
def __init__(self):
self.__name="xxxxx"
@property
def name(self):
return self.__name
@name.setter
def name(self,name):
```

```
self.__name=name
```

```
a=A()
print(a.name)
a.name="Kenpachi"
print(a.name)
'''
```

```
# with another class:
```

```
'''
class A:
def __init__(self):
self.__name="xxxxx"
@property
def name(self):
return self.__name
@name.setter
def name(self,name):
self.__name=name
class B(A):
def namee(self):
print(self.name)
```

```
##a=A()
##print(a.name)
##a.name="Kenpachi"
##print(a.name)
b=B()
b.namee()
b.name="Aizen"
b.namee()
'''
```

```
# _____-
# Abstraction:
```

```
# hiding the implementation part and showing the fuctionality to the user .
## by using abstaract class and interfaace we can achieve abstraction.
### by using interface we can achieve 100 % abstraction
#--- wheras using abstract class we can achive 0-100%
```

```
# all methods in abstract classes are abstract method
#- if and only if interface method
```

```
# else mixing of method and abstract method == abstract class
```

```
# interface
'''
```

```
from abc import ABC, abstractmethod
```

```
class A(ABC):
@abstractmethod # interface
def hall(self):
pass
@abstractmethod # if and only if it has
def bedrm(self):
```

```

pass
class B(A):    # if we didn't call any of abstract method it will be error
def hall(self):
print("hall 10X15 ft")
def bedrm(self):
print("bed_room 10X8 ft")
b=B()
b.hall()
b.bedrm()
"""

```

```

# abstract class
"""

```

```

from abc import ABC, abstractmethod

```

```

class A(ABC):
@abstractmethod
def hall(self):
pass
def bedrm(self):
pass
class B(A):    # here we didn't call bedrm cuz its not abstract method
def hall(self):
print("hall 10X15 ft")
b=B()
b.hall()
"""

```

```

#
"""

```

```

from abc import ABC , abstractmethod

```

```

class V(ABC):

@abstractmethod
def whl(self):
pass
@abstractmethod
def gear(self):
pass
def stand(self):
pass

```

```

class Car(V):
def whl(self):
print("4 wheels")
def gear(self):
print("5 gear")
class Bike(V):
def whl(self):
print("4 wheels")
def gear(self):
print("5 gear")
def stand(self):

```

```
print("stand avilable")
```

```
c=Car()
c.whl()
c.gear()
c.stand() # no error cuz it will pass on parrent abstract class
#
b=Bike()
b.whl()
b.gear()
b.stand()
'''
```

```
# argument passing
'''
```

```
from abc import ABC , abstractmethod
```

```
class V(ABC):
```

```
@abstractmethod
def whl(self):
pass
@abstractmethod
def gear(self):
pass
def stand(self):
pass
```

```
class Car(V):
def whl(self,w):
print(f"{w}")
def gear(self,g):
print(f"{g}")
class Bike(V):
def whl(self,w):
print(f"{w}")
def gear(self,g):
print(f"{g}")
def stand(self,s):
print(f"{s}")
```

```
c=Car()
c.whl(4)
c.gear(5)
c.stand() # no error cuz it will pass on parrent abstract class
#
b=Bike()
b.whl(4)
b.gear(5)
b.stand(2)
'''
```

```
#instance variable
```

```
# a variable defined within a class for which each created object(instance) has its own separate
```

```

copy
# direct
"""

class A:
def __init__(self):
self.x=10
a=A()
print(a.__dict__)
"""

# using argument and parameter
"""

class A:
def __init__(self,x):
self.x=x
a=A(10)
print(a.__dict__)
"""

# outside value
"""

class A:
def __init__(self):
pass
a=A()
print(a.__dict__)
a.x=10
print(a.__dict__)
"""

#
"""

class A:
def __init__(self,n):
self.n=n
print(self.n)
a=A(10)
"""

#
"""

class A:
def __init__(self,n):
self.n=n
def display(self):
print(self.n)
a=A(10)
a.display()
"""

# del
"""

class A:
def __init__(self,n):
self.n=n
del self.n
a=A(10)
print(a.__dict__)

```

```
"""
# del in another method
"""

class A:
def __init__(self,n):
self.n=n
def delete(self):
del self.n
a=A(10)
print(a.__dict__)
a.delete()
print(a.__dict__)
"""
```

```
"""
# outside del
"""

class A:
def __init__(self,n):
self.n=n
a=A(10)
print(a.__dict__)
del a.n
print(a.__dict__)
"""
```

```
#
```

```
#                                     Date: 04-04-2026
# saturday -- Mock test
#
```

```
#                                     Date: 05-04-2026
# holiday - easter
```

```
#
```

```
#                                     Date: 06-04-2026
```

```
# class level variable or static variable
```

```
# inside of class variable
"""
```

```
class A:
x=10
def func(self):
print("Yo")
a=A()
print(a.__dict__)
print(A.__dict__)
```

```
"""
# outside of the class declaring value
"""

class A:
def func(self):
print("Yo")
a=A()
A.x=10
print(a.__dict__)
print(A.__dict__)
"""
```

```
"""
# inside of method but not showing in op of A.__dict__
"""

class A:
def func(self):
x=10
print(x)
print("hi")
a=A()
print(a.__dict__)
print(A.__dict__)
"""
```

```
"""
# self became object of a
"""

class A:
def func(self):
self.x=10
print(x)
print("hi")
a=A()
print(a.__dict__)
print(A.__dict__)
"""
```

```
# A.x is inside of class
```

```
"""

class A:
def func(self):
A.x=10
print("hi")
a=A()
a.func()
print(a.__dict__)
print(A.__dict__)
"""
```

```
# class method
"""
```

```
class A:
@classmethod
def func(cls):
cls.x=10      # its not
```

```

print("yo")
a=A()
a.func()
print(a.__dict__)
print(A.__dict__)
'''

```

#

'''

```

class A:
    @classmethod
    def func(cls):
        A.x=10
        print("yo")
        a=A()
        a.func()
        print(a.__dict__)
        print(A.__dict__)
'''

```

#

```

#                                     Date: 07-04-2026
# static method only -- neither instance(object) nor class level(static) variable
'''

```

```

class A:
    @staticmethod
    def func():
        A.x=10
        print("hi")
        a=A()
        a.func()
        print(a.__dict__)
        print(A.__dict__)
'''

```

```

# access
# outside of the class
'''

```

```

class A:
    x=10 # it is outside
    def __init__(self):
        print(A.x)
        a=A()
        print(a.__dict__)
        print(A.__dict__)
'''

```

```

# outside the instance method
'''

```

```

class A:
    x=10
    def func(self):

```



```

print(A.x)
a=A()
a.func()
print(a.__dict__)
print(A.__dict__)
'''

```

```

## inside of the class method
'''

```

```

class A:
x=10
@classmethod
def func(cls):
print(cls.x)
a=A()
A.func()
'''

```

```

# inside the instance
'''

```

```

class A:
x=10
def func(self):
print(A.x)
a=A()
A.func()
'''

```

```

# inside the static method
'''

```

```

class A:
x=10
@staticmethod
def func():
print(A.x)
a=A()
a.func()
'''

```

```

### modify
## outside of the
# class
'''

```

```

class A:
x=10
a=A()
print(A.__dict__)
A.x=50
print(A.__dict__)
'''

```

```

# instance
'''

```

```

class A:
x=10
def func(self):

```

```

A.x=20
a=A()
print(A.__dict__)
a.func()
print(A.__dict__)
"""

# constructor

"""

class A:
x=10
def __init__(self):
A.x=50
a=A()
print(A.__dict__)
print(A.__dict__)
"""

# class method
# cls.x
"""

class A:
x=10
@classmethod
def func(cls):
cls.x=50
a=A() # summa or its not working
print(A.__dict__)
A.func()
print(A.__dict__)
"""

# A.x
"""

class A:
x=10
@classmethod
def func(cls):
A.x=50
a=A() ## this is summa or not working cuz there is no object
print(A.__dict__)
A.func()
print(A.__dict__)
"""

# static

"""

class A:
x=10
@staticmethod
def func():
A.x=50
a=A()
print(A.__dict__)
a.func()
print(A.__dict__)

```

```
'''
```

```
# ## delete
```

```
## outside of the
```

```
# class
```

```
'''
```

```
class A:
```

```
x=10
```

```
print(A.__dict__)
```

```
del A.x
```

```
print(A.__dict__)
```

```
'''
```

```
#
```

```
## inside of the
```

```
# constructor
```

```
'''
```

```
class A:
```

```
x=10
```

```
def __init__(self):
```

```
del A.x
```

```
print(A.__dict__)
```

```
a=A()
```

```
print(A.__dict__)
```

```
'''
```

```
# instance method
```

```
'''
```

```
class A:
```

```
x=10
```

```
def func(self):
```

```
del A.x
```

```
a=A()
```

```
print(A.__dict__)
```

```
a.func()
```

```
print(A.__dict__)
```

```
'''
```

```
# @classmethod
```

```
# A.x
```

```
'''
```

```
class A:
```

```
x=10
```

```
@classmethod
```

```
def func(cls):
```

```
del A.x
```

```
a=A()
```

```
print(A.__dict__)
```

```
A.func()
```

```
print(A.__dict__)
```

```
"""
```

```
# cls.x
```

```
"""
```

```
class A:
```

```
x=10
```

```
@classmethod
```

```
def func(cls):
```

```
del cls.x
```

```
a=A()
```

```
print(A.__dict__)
```

```
A.func()
```

```
print(A.__dict__)
```

```
"""
```

```
# @staticmethod
```

```
"""
```

```
class A:
```

```
x=10
```

```
@staticmethod
```

```
def func():
```

```
del A.x
```

```
a=A()
```

```
print(A.__dict__)
```

```
A.func()
```

```
print(A.__dict__)
```

```
"""
```

```
#
```

```
#
```

```
#
```

Date: 08-04-2026

```
#
```

```
#
```

Date: 09-04-2026

```
#
```

```
#
```

Date: 10-04-2026